

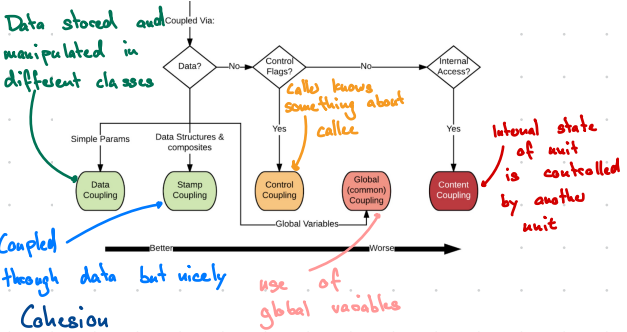
Week 07 & 08

Coupling:

→ Coupling is the property which indicates the strength of connections between different program elements → Strong coupling is problematic

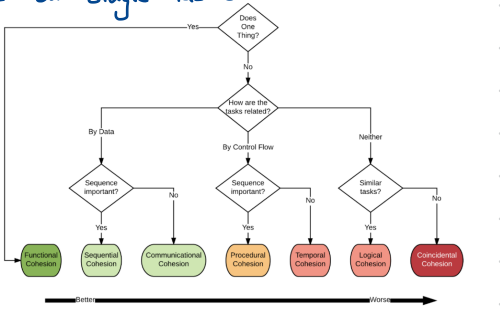
→ Coupling can be limited by:

- **Minimize the number of interfaces between elements:** The more interfaces two program elements need to share, the more tightly they are coupled to each other.
- **Minimize the complexity of interfaces:** Since some coupling is often needed, reducing the complexity of this coupling to its core elements can make the interaction between the program elements clearer and easier to reason about and evolve.
- **Avoid control flow coupling:** It can often be convenient to pass objects that control the flow of computation within another element. While this is ok if the element being passed is some type of data structure, it can be more problematic if the control flow is being influenced by simple control flow flags (e.g., some kind of **boolean** flag that takes one program path over another).



Cohesion

→ Cohesion describes, how focused single elements are on single tasks



SOLID Principles

1. **Single responsibility:** A software module should do one thing and do it well.

Many design patterns have been explicitly crafted to encourage designs that adhere to the single responsibility principle. For example:

- Strategy pattern: In this pattern, modules encapsulate algorithms; this means we create modules that *only* implement a specific algorithm.
- Command pattern: This pattern separates the notion of an action that can be performed from its implementation. This results in small modules that only provide the features needed for a specific action.
- State pattern: Systems often depend differently according to their internal state. Rather than having large modules that need to reason globally about all states, this pattern encapsulates the behaviours for a single state in a single module resulting in smaller, more targetted code.

2. **Open/closed:** Open to extension but closed to modification. One common code smell for violating the open/closed principle → *Easy to do with classes* are **instanceof** or **typeof** checks within the code.

3. **Liskov substitution:** Any object can be interchanged with any other object that has the same parent type. → *Eagle & Falcon both use bird.fly()*

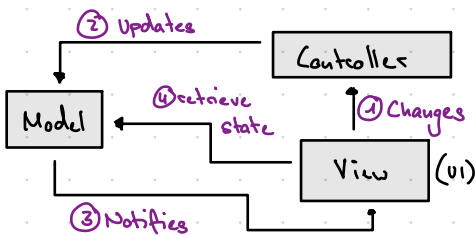
4. **Interface segregation:** Clients should not be forced to depend on interfaces they do not use. → *Separating interfaces based on client needs*

5. **Dependency inversion:** Classes should depend on abstractions, not implementations. This usually happens by injecting an interface between the two concrete classes and having the classes take dependencies on the interface instead.

Week 09

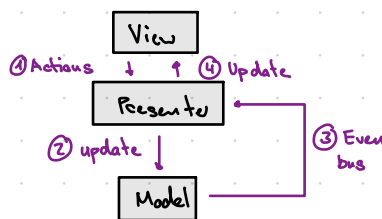
→ In Low-Level design we talk about how one design will be implemented

Model-View-Controller



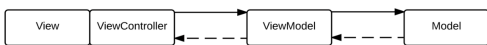
- ↳ Tries to split UI from business logic
- ↳ Controller lets Model know when change has happened in View

Model View Presenter



↳ In MVP the model uses an Eventbus which should make debugging easier

MVVM



↳ View Model contains the applications logic & View Controller prepares Data for the View

Microservice

→ Design Approach of developing a application as a suite of small services

Architectural Styles

Architectural styles minimize coupling. They result in a consistent set of principled techniques. They are resilient to change, they are a source of guidance, and they enable the reuse of established engineering knowledge. The following are architectural styles.

- Language Based: e.g., Object-oriented
- Dataflow: availability of data controls computation, e.g., Pipe-and-Filter
- Layered: hierarchically organized, e.g., Client Server
- Monolith vs Microservice

Week 10

Testing

	manual	automatic
empirical	manual testing	automated testing
analytical	inspection	program analysis

→ A defect is a problem in the code, a failure is the resulting behaviours

→ Unit Test: Input - Output behaviour of one function

→ Integration Test: Multiple functions, often at level of UI

→ Regression Test: Verifying that previously working code didn't break due to new feature

→ Smoke Test: Subset of integration tests to get over all since if system is working

→ System Test: Test part of the whole System

→ Analysis is using a logical rather than an empirical approach to spot problems

White box testing

Introduction to Coverage

Has your code been tested enough? Our goal is not the have a bug-free system, but

- 1) To ensure our code implements its specification.
- 2) To find defects in our implementation.
- 3) To understand where we can expect to have found defects.

Flow independent coverage: cheap and easy to reason about. (block/line/statement coverage)

Flow dependent coverage: (branch/path/MCC = multiple condition coverage)

Black box testing

Equivalence class partitioning⁶⁰

We don't see the code. We need to heavily **leverage the specifications** of the code to define the expected behavior. We cannot exhaustively test methods. Therefore, do **input partitioning** (design edge cases).

Input partitioning⁶¹

Build equivalence classes of input. Have one test for each equivalence class.

Output partitioning⁶²

Instead of input partitioning, we can design our test cases to make sure that we **exercise the range of outputs** that we expect to see.

Boundary value analysis⁶³

Besides valid inputs, also test what happens when you pass invalid/unexpected inputs to the function. Also test inputs that are right on the boundary between valid and invalid or on the boundary between two input partitions. Often the number zero is such an input. Testing the boundaries tests if our code is robust.

Refactoring

→ Refactoring means improving the structure of the code without adding new features

↳ brings some risks without substantial benefits for the customer

→ Bad Code Smells:

Bloaters = size of program elements make them hard to understand/change (long method, large class, long parameter list, primitive obsession).

OO Abusers: failure to leverage OO design (switch statements, refused bequest, message chain).

Change preventers: make it harder to evolve code (divergent changes, shotgun surgery).

Dispensable: unnecessary complexity (duplicate code, dead code, speculative generality, comments).

Couplers: unnecessary coupling (feature envy, middleman, inappropriate intimacy).

Week 11

Binary Compatibility: A compiled Program can run together with a compiled Program of an earlier version

Add API package	-	Binary compatible
Delete API package	-	Breaks compatibility
Add API type to API package	-	Binary compatible
Delete API type from API package	-	Breaks compatibility
Add non- public (non-API) type to API package	-	Binary compatible
Delete non- public (non-API) type from API package	-	Binary compatible
Change non- public (non-API) type in API package to make public (API)	-	Binary compatible
Change public type in API package to make non- public	-	Breaks compatibility
Change kind of API type (class, interface, enum, or annotation type)	-	Breaks compatibility (1)

Issue Tracking:

↳ Assign tasks and bugs to people

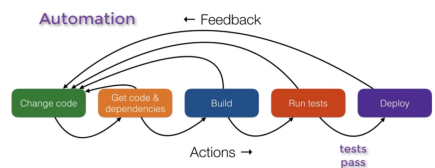
↳ A bug report consists of:

- Title (also called Summary, uniquely identify bug report, explain the problem not your suggested solution).
- Description (context, reproducible steps, actual and expected result)
- Status
- Assignee
- Priority
- Target Milestone
- Comments: might be only way of communication, document history/rationale of development process.
- Others

↳ Steps of fixing a bug:

Reproduce bug, examine exception in console, debug, locate error.

Software Automation:



Infrastructure as a Service (IaaS)

↳ On Demand access to Cloud based servers and so on

→ Amazon Compute Cloud

Platform as a Service (PaaS)

↳ On demand access to a cloud hosted Platform for Development

↳ For API development and management

→ Google App Engine

Software as a Service (SaaS)

↳ On demand access to ready-to-use Software on cloud

→ Google Docs

SaaS vs. PaaS vs. IaaS
SaaS
+ offloading all day-to-day management to the third-party vendor
- giving up all control over features and functionality, data storage, user access and security.
PaaS
+ build a custom application, control over features.
+ offload management of infrastructure and application
- responsibility for managing the application and associated data.
IaaS
+ build its own development platform and application.
+ complete control over operating systems and server configurations
- managing and maintaining systems, server, development platform and applications that run on them.

CI/CD

↳ A Solution to "Integration Hell"

DevOps = I think this means a team consisting of a developer team (coding) and an operations team (deployment/production)

Blue - Green Deployment:

Two easily changeable environments for customers / updates